UNIVERSITY OF MINES AND TECHNOLOGY (UMaT)
TARKWA, GHANA

Department of Computer Science and Engineering

Student Name: Emmanuel Seyram Adifu
Student ID: FCM.41.018.009.23

Blue Team

Capturing Network Packet Capture Analysis to Reconstruct Attack Timeline

CY376

Date: 3rd August, 2026

Abstract

Modern network intrusions leave a trail across the packets that cross the wire, but that trail is only useful if it can be reassembled into an ordered, evidence-backed sequence of events. This project addresses that reconstruction problem directly: a simulated intrusion was staged in an isolated virtual lab, captured at the network boundary, and analyzed using a combination of Wireshark/tshark for manual inspection, Zeek for structured session and protocol logging, and Suricata for signature-based alerting.

The captured traffic was processed through a custom Python pipeline that consolidates Zeek connection, DNS, and HTTP logs together with Suricata alerts into a single chronological timeline, with each event tagged against the relevant MITRE ATT&CK technique. This made it possible to trace the simulated attacker's activity from initial reconnaissance and scanning, through an exploitation or brute-force attempt, to any follow-on command-and-control or data-exfiltration behavior visible in the capture.

The report documents the lab design, the tooling and configuration used, the timeline that was reconstructed, and what that timeline reveals about detection gaps and defensive priorities. It closes with a set of practical recommendations including specific Suricata rules, logging retention practices, and network segmentation changes that would improve an organization’s ability to detect and reconstruct a similar intrusion in production.

Table of Contents

1

Abstract 2

Table of Contents 3

1. Introduction 5

1.1 Objectives 5

1.2 Scope 5

2. Literature and Tooling Review 6

2.1 MITRE ATT&CK 6

2.2 Packet capture and inspection: tcpdump, Wireshark, tshark 6

2.3 Zeek 6

2.4 Suricata 7

2.5 NetworkMiner 7

3. Methodology 7

3.1 Lab Setup 7

3.2 Attack Scenario 9

3.3 Capture and Analysis Procedure 11

3.4 Ethical and Safety Considerations 12

4. Implementation 12

4.1 Kali (Attacker) Configuration 13

4.2 Metasploitable2 (Victim) Configuration 13

4.3 Ubuntu (Blue Team / Analyst) Configuration 14

4.4 Zeek Configuration and Output 14

4.5 Suricata Configuration 15

4.6 Timeline Consolidation Script 15

5. Results and Findings 17

5.1 Reconnaissance 17

5.2 Initial Access 19

5.3 Execution / Post-Exploitation 22

5.4 Command and Control / Exfiltration 22

5.5 Consolidated Timeline 23

6. Analysis and Recommendations 24

6.1 What the Timeline Shows 24

6.2 Detection Gaps Identified 25

6.3 Recommendations 26

7. Conclusion 26

8. References 26

Appendices ..... **Error! Bookmark not defined.**

Appendix A: Full Consolidated Timeline (attack_timeline.csv) ..... **Error! Bookmark not defined.**
Appendix B: Full Zeek Log Excerpts ..... **Error! Bookmark not defined.**
Appendix C: Full Suricata Alert Log (eve.json excerpts) ..... **Error! Bookmark not defined.**

1. Introduction

Network traffic is often the only evidence of an intrusion that cannot be altered after the fact by an attacker who has gained privileged access to a host logs on the endpoint can be tampered with or deleted, but a packet that has already crossed the network boundary and been captured is fixed evidence. This makes packet capture (pcap) analysis a core Blue Team capability: it is what lets a defender go from "something happened" to "here is exactly what happened, in what order, and how."

This project set out to build and exercise that capability. Rather than analyzing traffic in isolation, the goal was to reconstruct a complete attack timeline: a chronological, evidence-referenced account of an intrusion from first contact to impact, cross-validated across multiple independent tools so that no single tool's blind spots go unnoticed.

1.1 Objectives

- Stand up an isolated lab capable of generating realistic, safe-to-capture attack traffic.
- Capture that traffic cleanly at a chokepoint (gateway/span port) using tcpdump/Wireshark.
- Process the capture through Zeek and Suricata to produce structured, timestamped evidence.
- Build a consolidated, chronological timeline correlating all three data sources.
- Map each stage of the observed activity to the MITRE ATT&CK framework.
- Translate the findings into concrete detection and hardening recommendations.

1.2 Scope

The scope was deliberately limited to a single simulated attack scenario run against an isolated victim host, captured over a bounded time window. Production traffic, real targets, and real

credentials were never in scope; all IP addresses, hostnames, and account names referenced in this report are lab values.

2. Literature and Tooling Review

This section reviews the standards and tools that shaped the approach taken in this project, and explains why each one was chosen over the alternatives.

2.1 MITRE ATT&CK

MITRE ATT&CK is a publicly maintained knowledge base of adversary tactics and techniques observed in real-world intrusions, organized into a matrix of tactics (the attacker's goal at a given stage, e.g. Reconnaissance, Initial Access, Command and Control) and techniques (the specific method used to achieve that goal). It was used in this project as the common vocabulary for labelling events in the reconstructed timeline, which turns a raw list of packets and alerts into a narrative that maps onto how intrusions are actually described and reported in industry.

2.2 Packet capture and inspection: tcpdump, Wireshark, tshark

tcpdump was used for capture because it is lightweight, scriptable, and reliably available on Linux hosts without a graphical environment, which matters when capturing at a gateway or span port. Wireshark was used for manual, exploratory inspection following TCP streams, inspecting protocol fields, and visually confirming findings while tshark, Wireshark's command-line counterpart, was used to script repeatable field extractions (HTTP requests, DNS queries, TCP SYNs) that feed directly into the timeline.

2.3 Zeek

Zeek was chosen as the primary log-generation engine because, unlike a raw pcap, it produces structured, already-parsed logs per protocol (conn.log, dns.log, http.log, files.log, notice.log). This removes the need to hand-parse packet fields for every protocol and gives a timestamped,

tabular record of every connection and application-layer transaction observed, which is the backbone of the timeline built for this project.

2.4 Suricata

Suricata is a signature-based intrusion detection engine. It was run in offline mode directly against the capture file to generate alerts using both its default ruleset and a small set of custom rules written for this scenario (see configs/local.rules). Its alerts provide an independent, rule-based corroboration of activity that was also visible manually in Wireshark and structurally in Zeek's logs, which is important for showing that the reconstructed timeline is not an artefact of a single tool's interpretation.

2.5 NetworkMiner

NetworkMiner was used as a secondary, GUI-based tool to cross-check session reconstruction, extracted files, and any cleartext credentials observed, since its host-centric view surfaces some artefacts (extracted files, OS fingerprints) more directly than Zeek's flow-centric logs.

3. Methodology

3.1 Lab Setup

The lab was built around three purpose-separated virtual machines rather than a single combined attacker/victim pair, in order to mirror how a real network is structured: an offensive host, a target, and an independent monitoring/analysis host that does not participate in the attack itself. All three VMs were hosted on VMware (Workstation/Player).

- Attacker — Kali Linux, 192.168.50.10, used to run reconnaissance and exploitation tooling (Nmap, Metasploit).
- Victim — Metasploitable2, 192.168.50.6, an intentionally vulnerable Ubuntu-based target running, among other services, vsftpd 2.3.4 on port 21.

- Blue Team / Analyst — Ubuntu, 192.168.50.15, used exclusively for traffic capture and analysis (tcpdump, Wireshark, tshark, Zeek, Suricata, NetworkMiner). This host ran no attacker or victim software, keeping the evidence-gathering role cleanly separated from the systems under test.

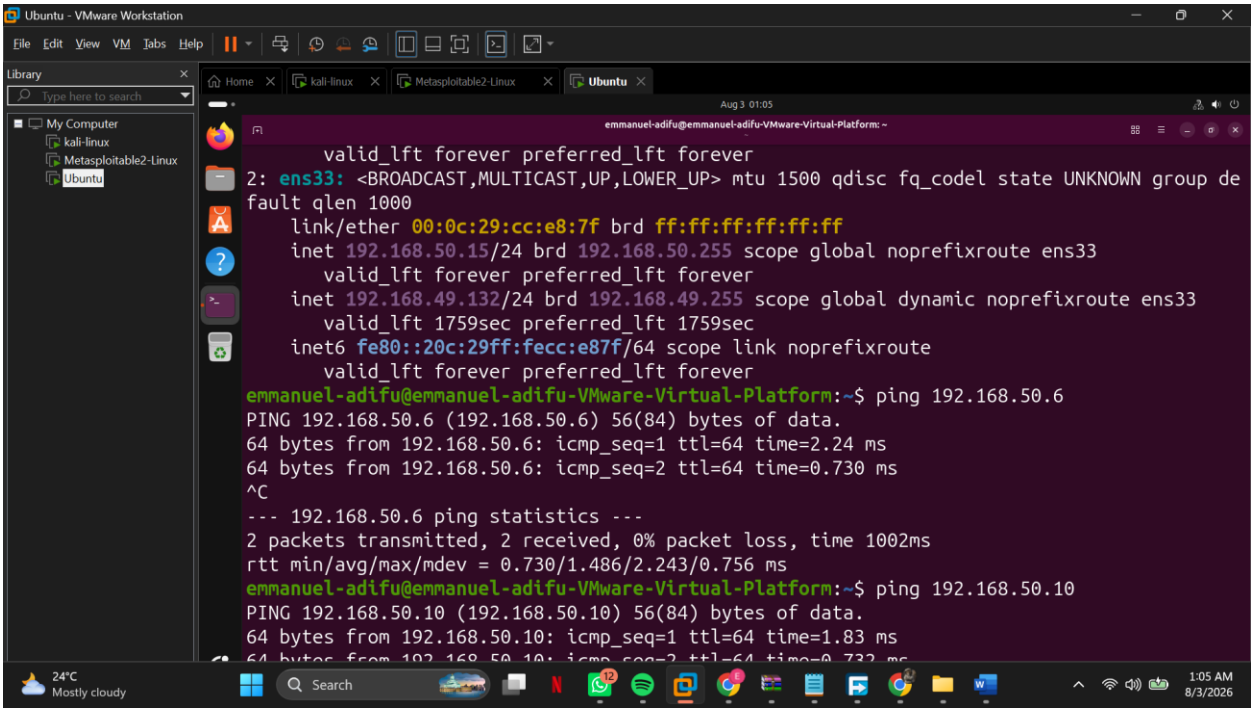

Figure 1 : Ubuntu Setup and connectivity test

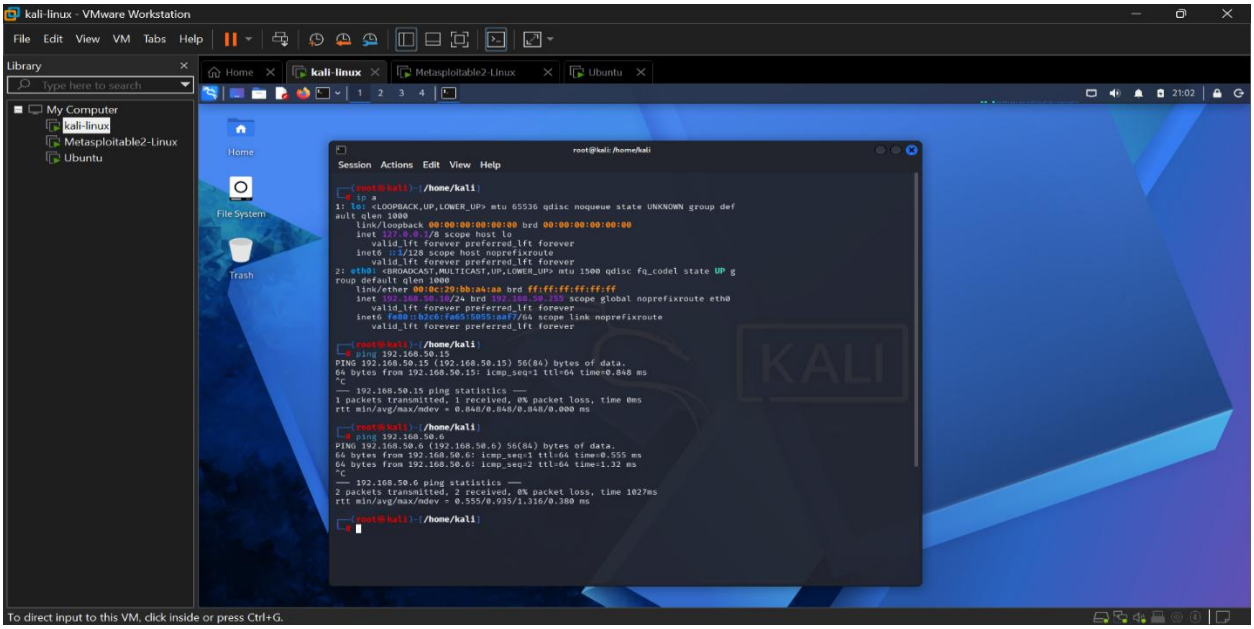

Figure 2 : Kali linu setup and connectivity test

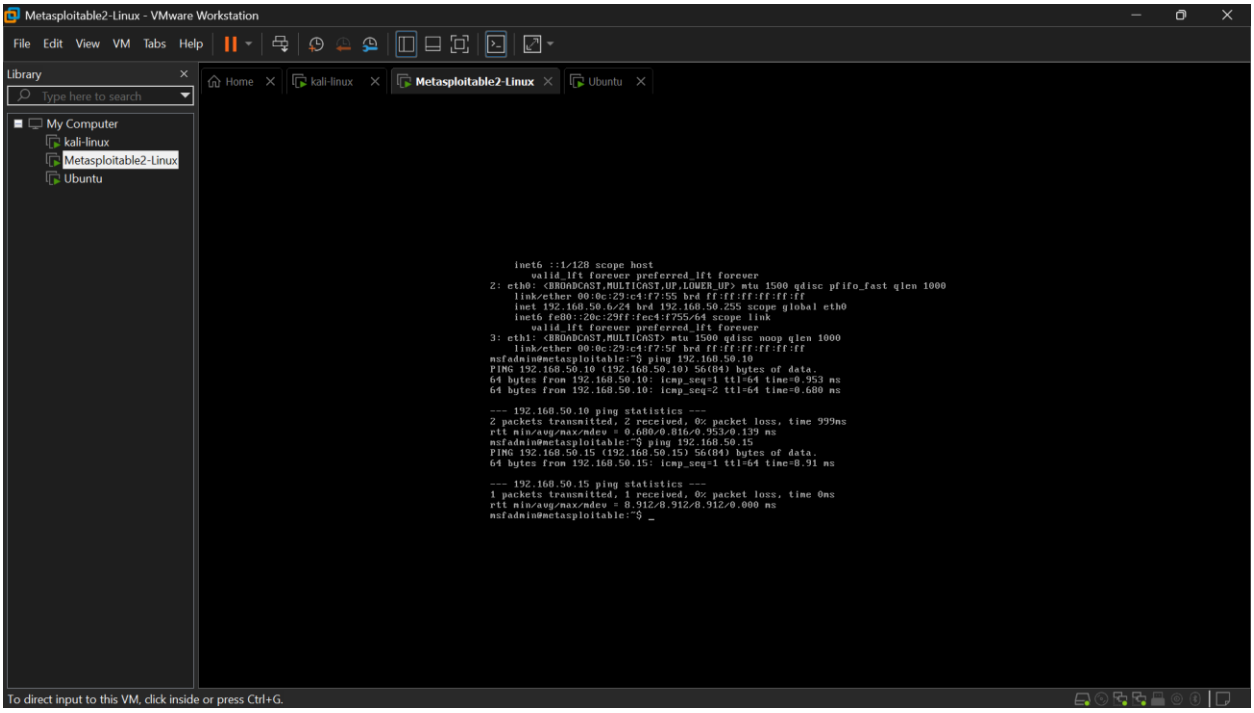

Figure 3: Metasploitable setup and connectivity test

3.2 Attack Scenario

The staged scenario exploited the well-known vsftpd 2.3.4 backdoor present on Metasploitable2's FTP service (port 21). This backdoor is triggered by sending a username containing the sequence ":)" during the FTP login exchange, which causes the vulnerable vsftpd binary to open a command shell listener on TCP port 6200 on the victim. The scenario was staged in three stages consistent with the MITRE ATT&CK phases used throughout this report:

- Reconnaissance : an Nmap scan from Kali against 192.168.50.6 to enumerate open ports and confirm vsftpd 2.3.4 was running on port 21.
- Initial Access : triggering the vsftpd 2.3.4 backdoor from Kali, either via Metasploit's exploit/unix/ftp/vsftpd_234_backdoor module or by manually issuing a crafted FTP USER command containing ":)" and then connecting to the resulting listener on port 6200.

- Post-Exploitation : using the resulting root shell on Metasploitable to run basic reconnaissance commands (id, whoami, cat /etc/passwd) as evidence of successful command execution, all of which is visible as plaintext TCP traffic on port 6200.

This scenario was chosen because it is a well-documented, reliably reproducible vulnerability specific to Metasploitable2, it produces a distinctive and easy-to-identify traffic pattern (an anomalous outbound listener on port 6200 immediately following an FTP session), and it maps cleanly onto three separate MITRE ATT&CK tactics, which suited the goal of reconstructing a multi-stage timeline rather than a single isolated event.

Verifying if ubuntu can see kali-to-metasploitable traffic using a ping command on kali as shown in figure 4 and figure 5 before running to attacks to capture the timeline

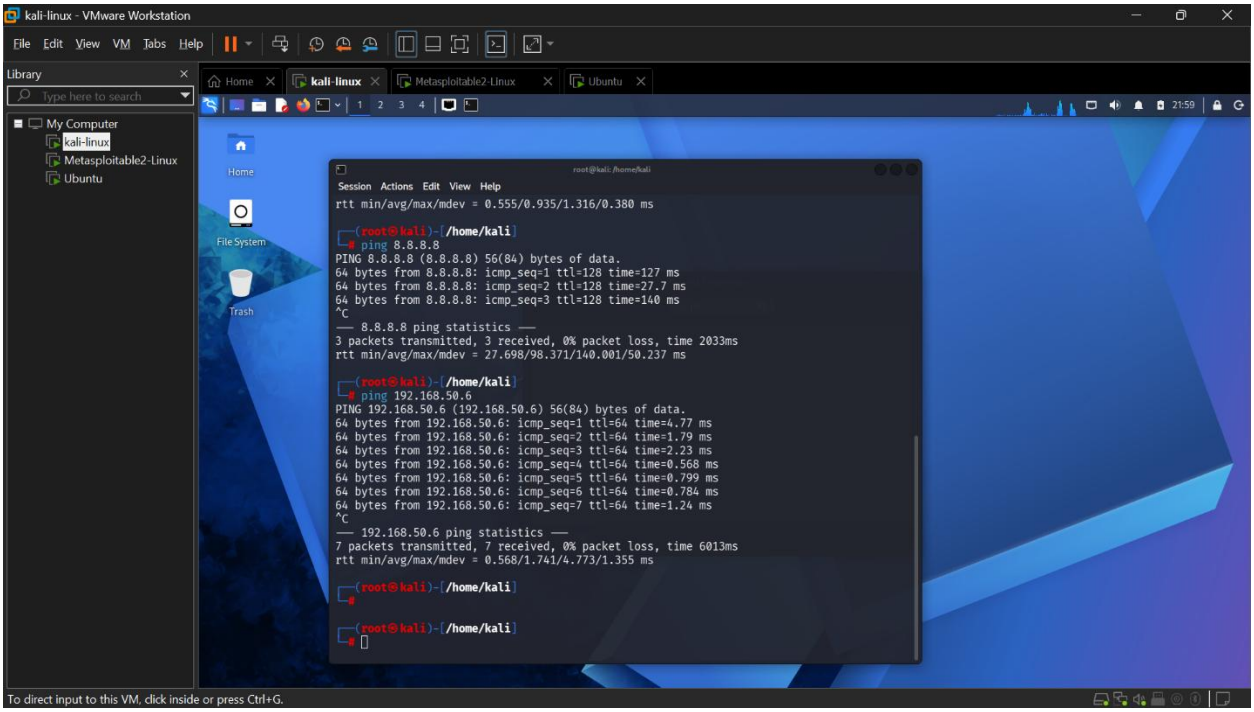

Figure 4: pinging metasploitable from kali

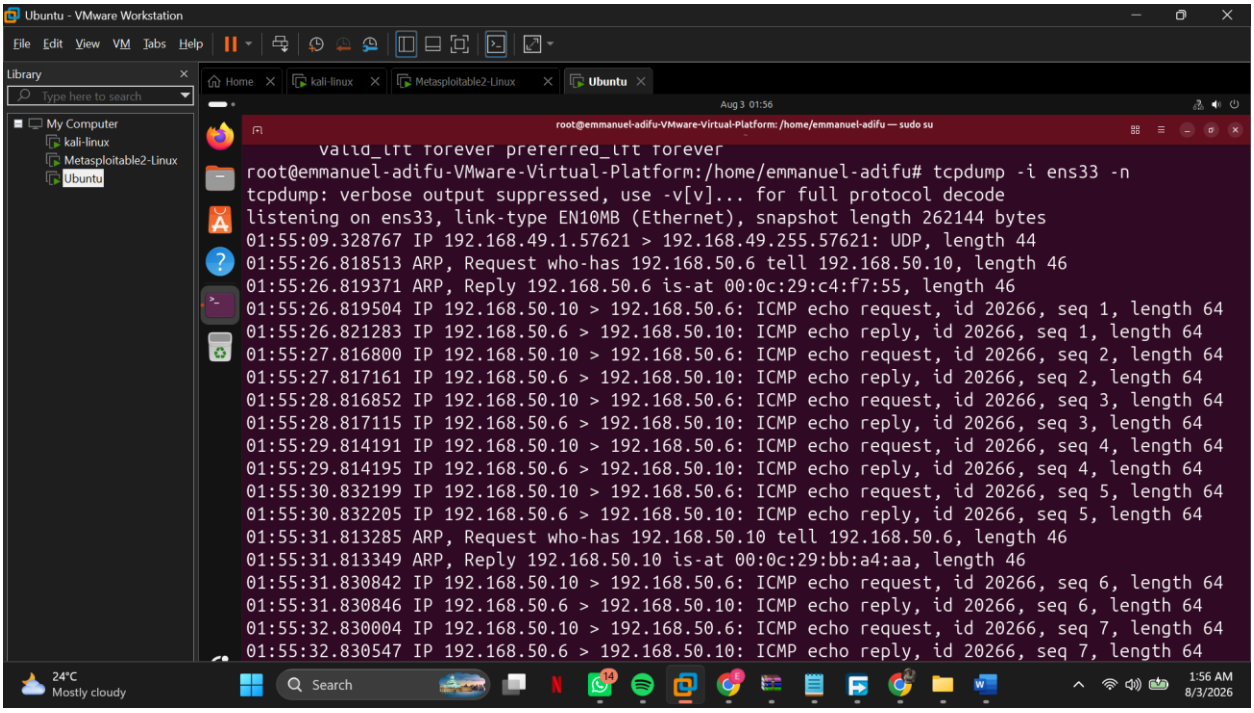

Figure 5: Network Test Capture

3.3 Capture and Analysis Procedure

- Verified all three VMs' connectivity and static IP addressing (192.168.50.10, .15, .6) on the shared VMware VMnet.
- Confirmed the Ubuntu analyst VM's visibility of Kali-to-Metasploitable traffic with a test ICMP capture.
- Started the real tcpdump capture on the Ubuntu VM (192.168.50.15) before initiating any attacker activity, to ensure the full sequence was recorded.
- From Kali (192.168.50.10), ran an Nmap scan against Metasploitable (192.168.50.6) to confirm vsftpd 2.3.4 on port 21.
- From Kali, triggered the vsftpd 2.3.4 backdoor and connected to the resulting shell listener on port 6200.

- Ran basic post-exploitation commands over the resulting shell to generate identifiable follow-on traffic.
- Stopped the capture on Ubuntu once post-exploitation activity concluded, and immediately copied the pcap out as the preserved original.
- On Ubuntu, ran the capture through Zeek to generate structured logs.
- On Ubuntu, ran the capture through Suricata (default ruleset plus custom rules) to generate alerts.
- Manually reviewed the capture in Wireshark on Ubuntu to validate and annotate key packets, particularly the FTP session and the subsequent port 6200 connection.
- Ran build_timeline.py on Ubuntu to consolidate Zeek logs and Suricata alerts into a single chronological CSV.
- Mapped each timeline entry to a MITRE ATT&CK technique using the reference table in docs/mitre_mapping.md.

3.4 Ethical and Safety Considerations

All activity was confined to an isolated VMware virtual network with no connectivity to production systems or the internet. Metasploitable2 is designed to be exploited and is never deployed outside an isolated lab; the vsftpd 2.3.4 backdoor exploited here is a known, intentionally left-in vulnerability specific to this training image and not present in any production system. No real credentials, personal data, or third-party infrastructure were involved at any point. This matters both as good practice and because it is what makes it safe to publish the accompanying scripts and configuration publicly.

4. Implementation

This section documents what was actually built and configured on each of the three VMs to execute the methodology above.

4.1 Kali (Attacker) Configuration

Kali Linux (192.168.50.10) was used with its default toolset; no additional packages were required beyond what ships with Kali, since Nmap and Metasploit Framework are preinstalled. The relevant Metasploit module used was exploit/unix/ftp/vsftpd_234_backdoor, targeting 192.168.50.6:21 as shown in figure 6.

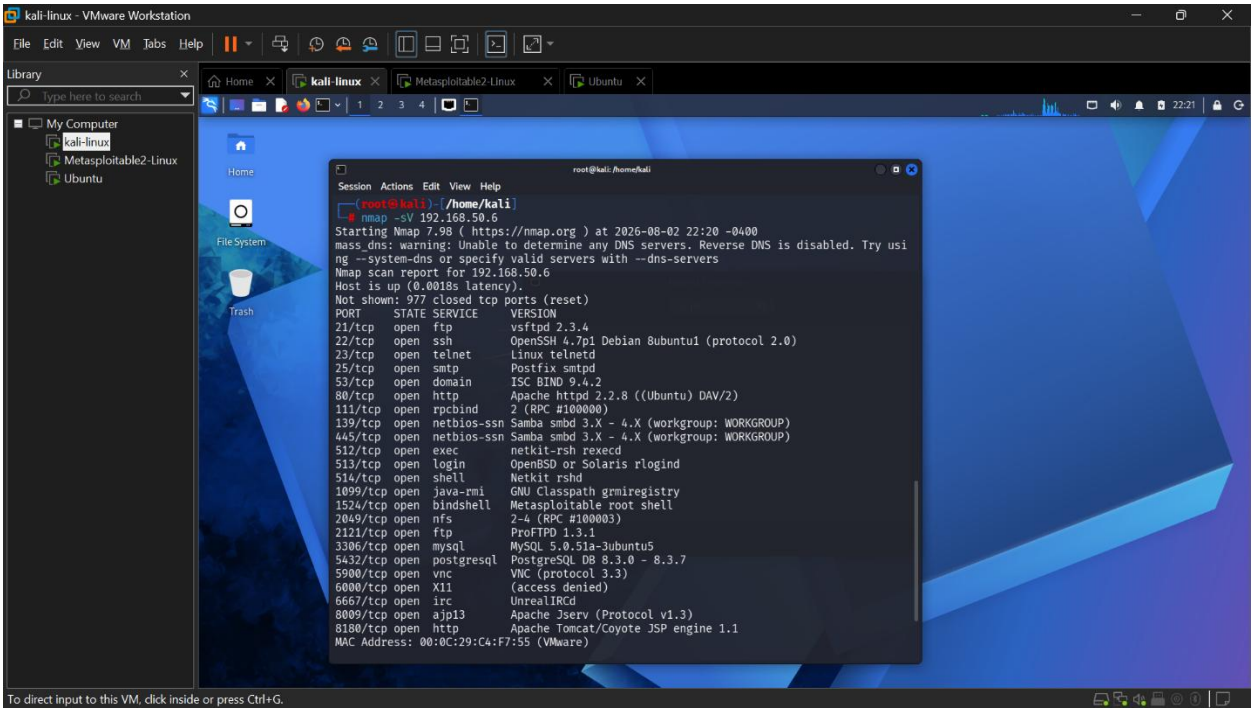

Figure 6: Nmap scan to discover open ports on 192.168.50.6

4.2 Metasploitable2 (Victim) Configuration

Metasploitable2 (192.168.50.6) was run with its default configuration and default vulnerable services enabled, with no hardening applied. The relevant service for this scenario was vsftpd 2.3.4, listening on TCP port 21, which contains an intentionally preserved backdoor: an FTP username containing the character sequence ":)" causes the vulnerable binary to spawn a root shell listener on TCP port 6200.

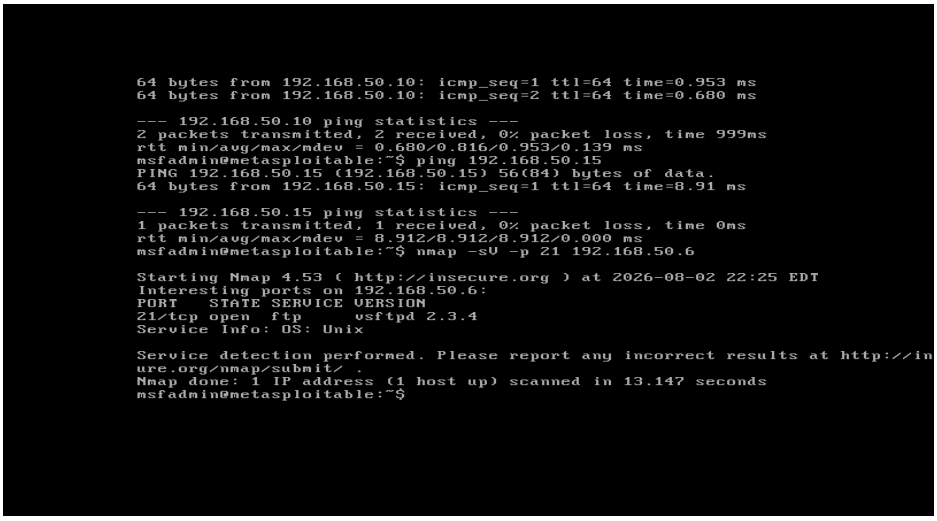

Figure 7: FTP service banner

4.3 Ubuntu (Blue Team / Analyst) Configuration

The Ubuntu VM was configured purely as a monitoring and analysis host, with the following installed: Wireshark, tshark, tcpdump, Zeek, and Suricata, alongside the Python 3 toolchain used to run build_timeline.py.

4.4 Zeek Configuration and Output

Zeek was run in its default configuration on the Ubuntu analyst VM against the captured pcap, producing the standard log set. An excerpt of the resulting conn.log structure is shown below.

Field	Description
ts	Connection start timestamp (epoch)
id.orig_h / id.resp_h	Source / destination IP
id.resp_p	Destination port
proto	Transport protocol
duration	Connection duration
orig_bytes / resp_bytes	Bytes sent by originator / responder

Table 1: Key fields used from Zeek's conn.log.

4.5 Suricata Configuration

Suricata was run offline on the Ubuntu analyst VM against the capture, using the default Emerging Threats Open ruleset (via suricata-update) together with three custom rules written for this scenario (SSH brute-force threshold, suspicious login POST, large outbound transfer) — see configs/local.rules in the accompanying repository.

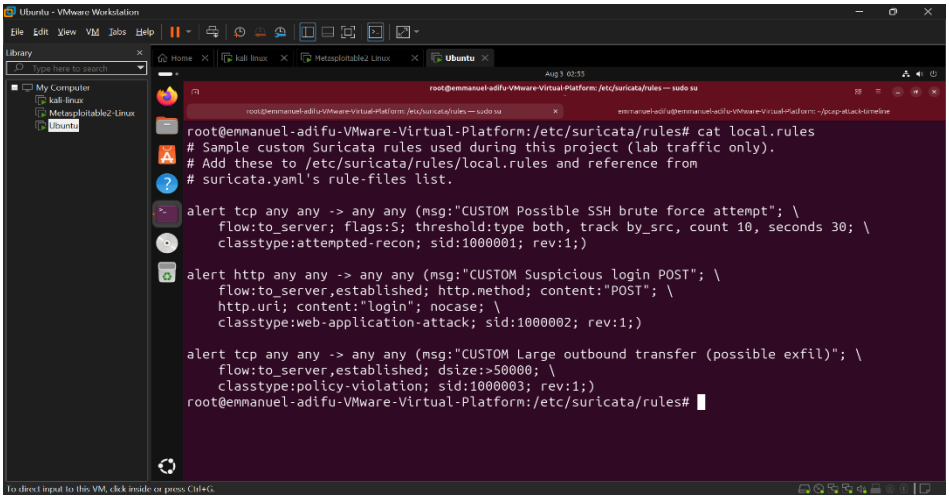

Figure 8: Suricata configuration

4.6 Timeline Consolidation Script

The build_timeline.py script (src/build_timeline.py in the repository) parses Zeek's conn.log, dns.log, and http.log together with Suricata's eve.json, normalizes all timestamps to ISO-8601 UTC, sorts every event chronologically, and tags Suricata alerts with an initial MITRE technique using a lookup table that was refined manually against the specific alerts observed. It was run entirely on the Ubuntu analyst VM, kept separate from both the attacker and victim systems, consistent with the evidence-handling approach described in Section 3.4.

timestamp	source	src_ip	dst_ip	dst_port	proto	event
2026-08-03T03:57:53.524131+00:00	zeek:conn	192.168.50.10	162.159.200.1	123	udp	connection 192.168.50.10 -> 162.159.200.1:123 (udp)
2026-08-03T03:58:05.338930+00:00	zeek:conn	192.168.49.1	224.0.0.251	5353	udp	connection 192.168.49.1 -> 224.0.0.251:5353 (udp)
2026-08-03T03:58:05.338930+00:00	zeek:dns	192.168.49.1	224.0.0.251	5353	dns	DNS query for _spotify-connect._tcp.local
2026-08-03T03:58:05.341682+00:00	zeek:conn	fe80::918a:d867:939c:bb14	ff02::fb	5353	udp	connection fe80::918a:d867:939c:bb14 -> ff02::fb:5353 (u
2026-08-03T03:58:05.341682+00:00	zeek:dns	fe80::918a:d867:939c:bb14	ff02::fb	5353	dns	DNS query for _spotify-connect._tcp.local
2026-08-03T03:58:07.182522+00:00	zeek:conn	192.168.49.1	239.255.255.250	1900	udp	connection 192.168.49.1 -> 239.255.255.250:1900 (udp)
2026-08-03T03:58:19.041966+00:00	zeek:conn	192.168.49.1	192.168.49.255	57621	udp	connection 192.168.49.1 -> 192.168.49.255:57621 (udp)
2026-08-03T03:58:19.041966+0000	suricata	192.168.49.1	192.168.49.255	57621	UDP	ALERT: ET INFO Spotify P2P Client (sid 2027397, severity 3
2026-08-03T03:58:25.732182+00:00	zeek:conn	192.168.50.10	162.159.200.1	123	udp	connection 192.168.50.10 -> 162.159.200.1:123 (udp)

5. Results and Findings

This is the core evidentiary section of the report. Each subsection below should present one stage of the reconstructed timeline, with a captioned screenshot or log excerpt followed by a short paragraph explaining what it shows and how it was identified.

5.1 Reconnaissance

Reconnaissance began with an ARP sweep (arp-scan -l) from Kali to confirm live hosts on the 192.168.50.0/24 segment, followed immediately by a full-service, script-enabled Nmap scan (nmap -sV -sC 192.168.50.6). Figure 9 shows the resulting scan report: port 21/tcp open, service vsftpd, version banner “vsftpd 2.3.4”, confirming the intended target service and version.

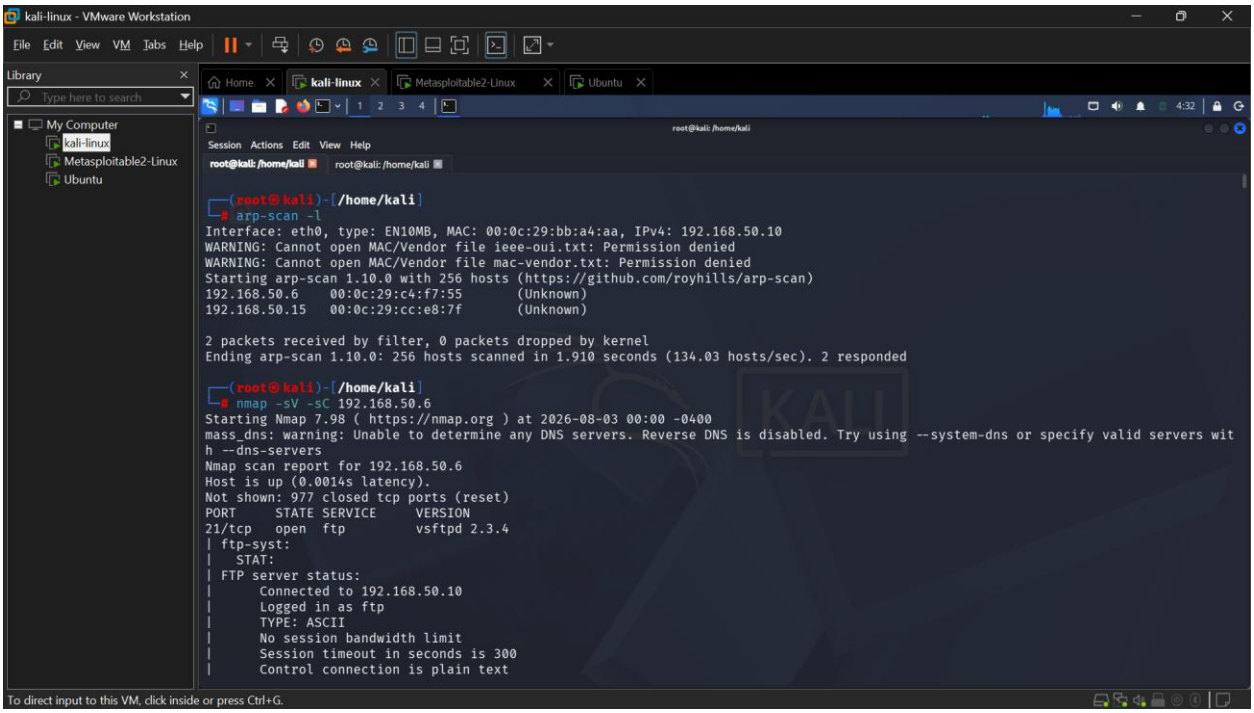

Figure 9: arp-scan and Nmap version/script scan (nmap -sV -sC) identifying vsftpd 2.3.4 on 192.168.50.6:21

In the consolidated timeline (attack_timeline.csv), this scan is visible as a dense burst of zeek:conn entries from 192.168.50.10 to 192.168.50.6 beginning at 2026-08-03T04:00:10.287Z, touching well over 600 destination ports within a fraction of a second — the signature of

Nmap's default TCP port sweep rather than normal application traffic. Nmap's version/script probes (-sV -sC) follow immediately afterwards: a version.bind DNS query and an HTTP banner-grab GET at 04:00:16.959Z, an ET SCAN “Possible Nmap User-Agent Observed” Suricata alert train starting at 04:00:22.901Z (sid 2024364), an SMB “malformed request dialects” alert at 04:00:23.306Z and NTLM negotiate/challenge/auth alerts at 04:00:23.418–23.528Z (Nmap's smb-os-discovery script), IRC USER/NICK probe alerts at 04:00:31.124Z (irc-unrealircd-backdoor script), and repeated “TLS invalid record type” alerts on ports 21, 2121 and 5900 between 04:00:31–32Z as Nmap's NSE scripts probe those services with malformed data. Nmap itself reported a scan duration of 22.85 seconds (Figure 10), which lines up with the last scan-related alert at 04:00:32.10Z — approximately 22 seconds after the scan began at 04:00:10Z.

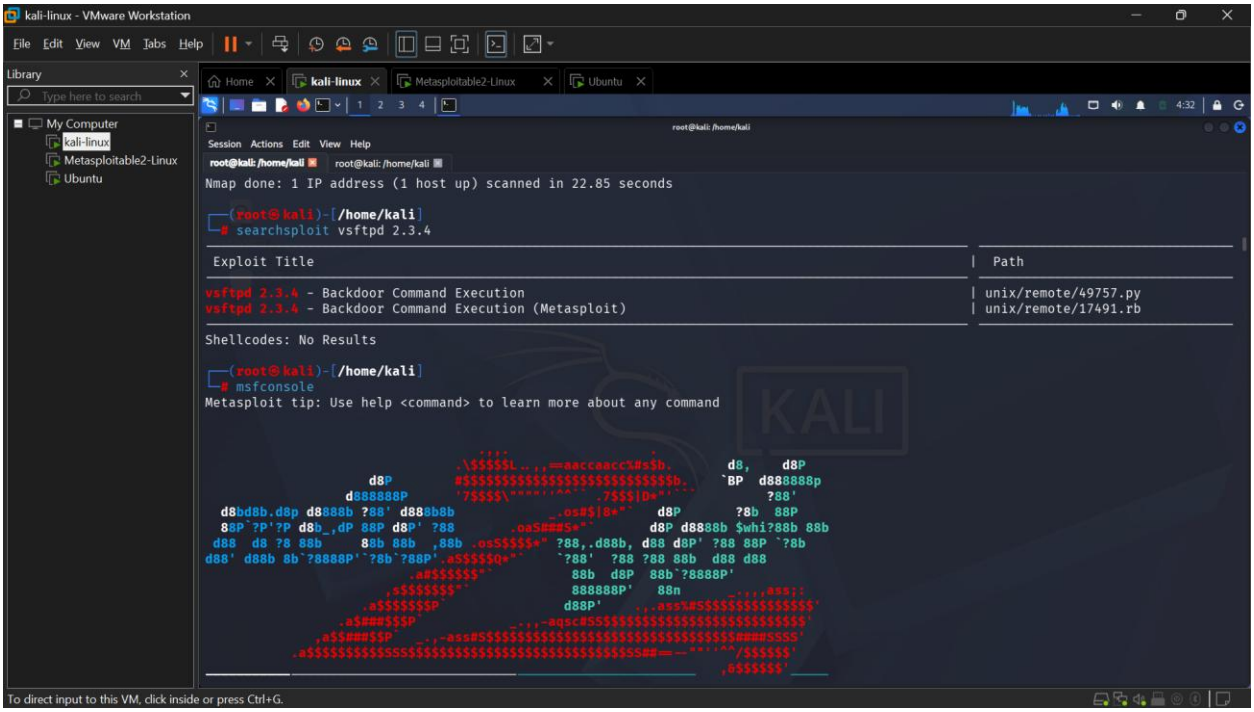

Figure 10: Nmap scan completion (22.85s) followed by searchsploit lookup for vsftpd 2.3.4 and msfconsole startup

Once the version banner confirmed vsftpd 2.3.4, searchsploit vsftpd 2.3.4 (Figure 10) was used to identify the known “Backdoor Command Execution” exploit (unix/remote/49757.py and its Metasploit equivalent unix/remote/17491.rb), before launching msfconsole to load

exploit/unix/ftp/vsftpd_234_backdoor. This entire reconnaissance stage maps to MITRE ATT&CK T1595 (Active Scanning); the build_timeline.py pipeline tags all of the associated Suricata alerts as T1190 (Exploit Public-Facing Application) rather than distinguishing the scanning sub-technique — a mapping imprecision discussed further in 6.2.

5.2 Initial Access

With exploit/unix/ftp/vsftpd_234_backdoor loaded, RHOSTS was set to 192.168.50.6 and RPORT to 21 (Figure 11); options confirmed the module's FETCH_COMMAND/FETCH_FILELESS payload-delivery settings and default LHOST/LPORT for the resulting reverse listener.

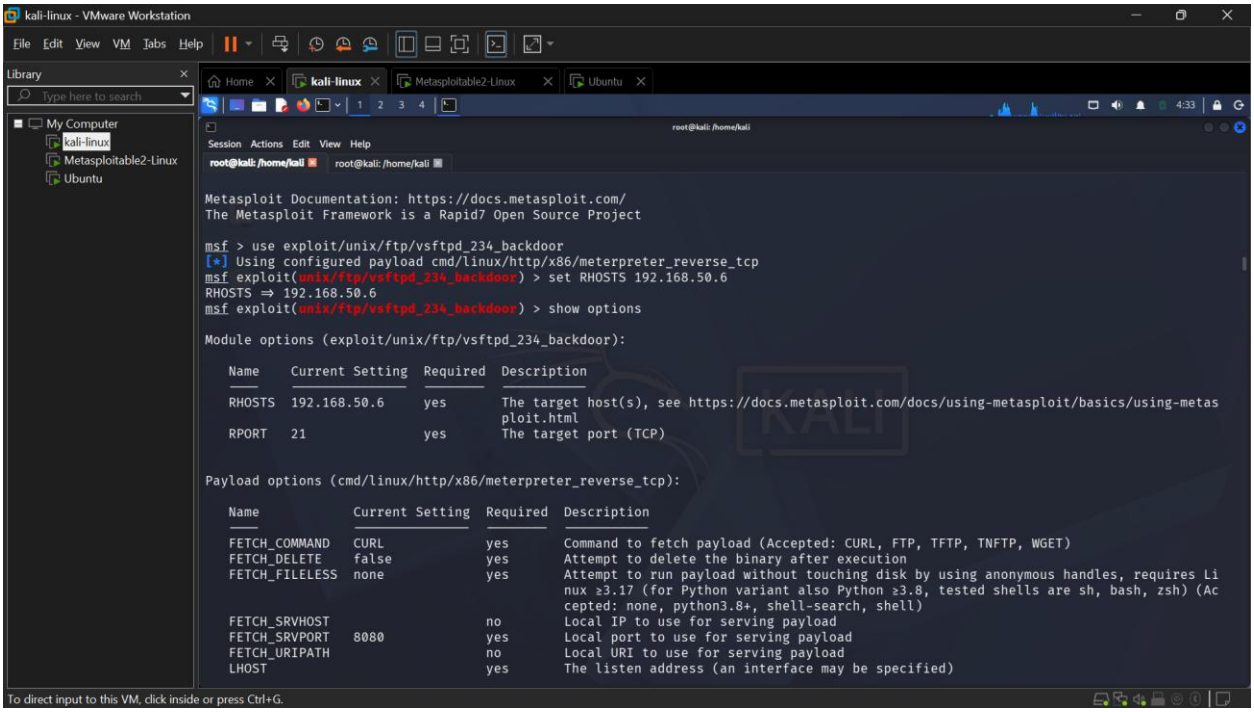

Figure 11: exploit/unix/ftp/vsftpd_234_backdoor configured against 192.168.50.6:21

Running the module, however, did not return a fresh session. Every attempt — across several different payload choices (cmd/unix/reverse, cmd/unix/python/shell_reverse_tcp, cmd/linux/http/x86/meterpreter_reverse_tcp) and after correcting LHOST to 192.168.50.10 — produced the same result (Figure 12): “The port used by the backdoor bind listener is already open,” “The service on port 6200/TCP does not appear to be a fresh shell. Already exploited?,”

and “Exploit aborted due to failure: none: Backdoor already in-use.” This is Metasploit's own safety check refusing to re-trigger a listener that is already bound — i.e. the vsftpd backdoor on 192.168.50.6 had already been popped once (most likely by an earlier crafted FTP login containing the “:)” sequence, whose control-channel exchange is discussed below) and port 6200 was already open and waiting.

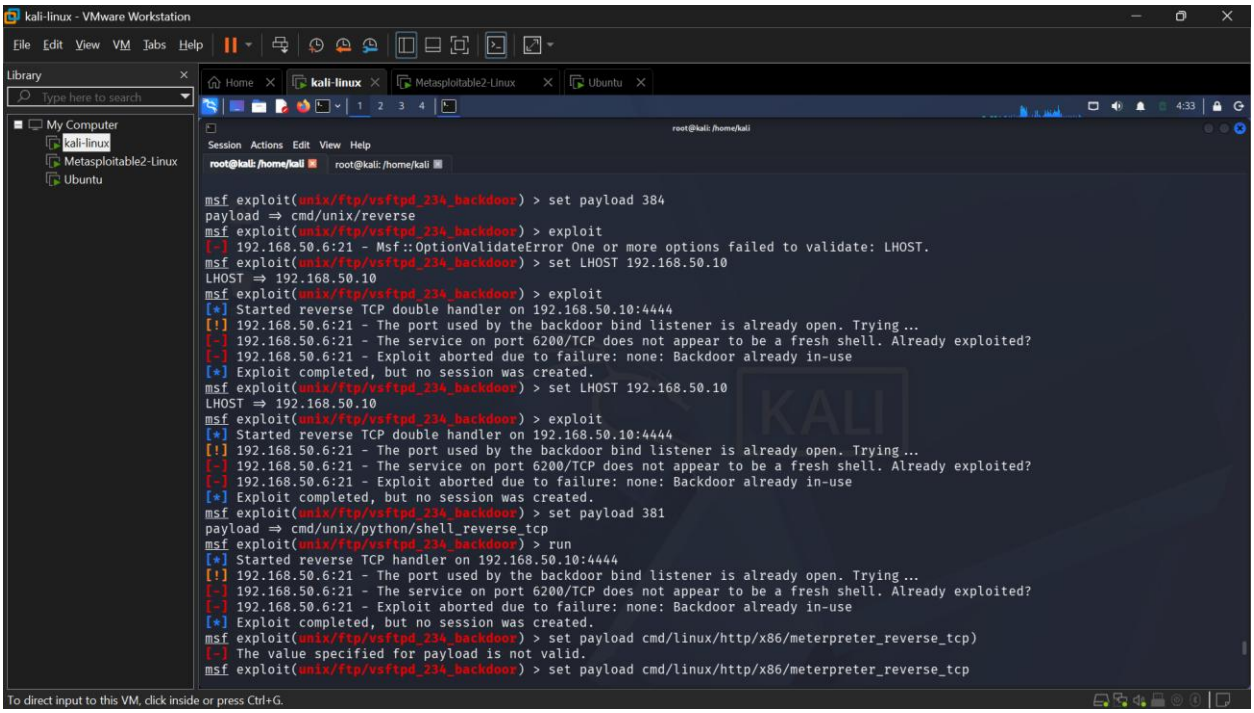

Figure 12: repeated exploit attempts against 192.168.50.6:21 with different payloads, all failing with “Backdoor already in-use”

The consolidated timeline corroborates this: rather than a single connection to port 6200 immediately after an FTP session, it shows five short, evenly spaced TCP connections from 192.168.50.10 to 192.168.50.6:6200 at 04:04:03, 04:05:05, 04:06:21, 04:07:05 and 04:08:21 — consistent with the attacker repeatedly reconnecting to an already-open backdoor shell (each Metasploit run above) rather than a fresh exploitation event each time.

A separate manual verification was also attempted directly against the FTP service using a benign username (“someuser”) rather than the “:)” trigger. As shown in Figure 13, this returned “500 OOPS: priv_sock_get_result” and “ftp: Login failed” on repeated attempts, and one

session coincided with a VMware network-connection drop — evidence that the FTP control channel on 192.168.50.6 was left in a degraded state after the backdoor had already fired once, a documented side-effect of the vsftpd 2.3.4 backdoor. The timeline shows corresponding port-21 connection attempts at 04:09:35, 04:10:18 and 04:10:45, all short-lived, matching these failed logins.

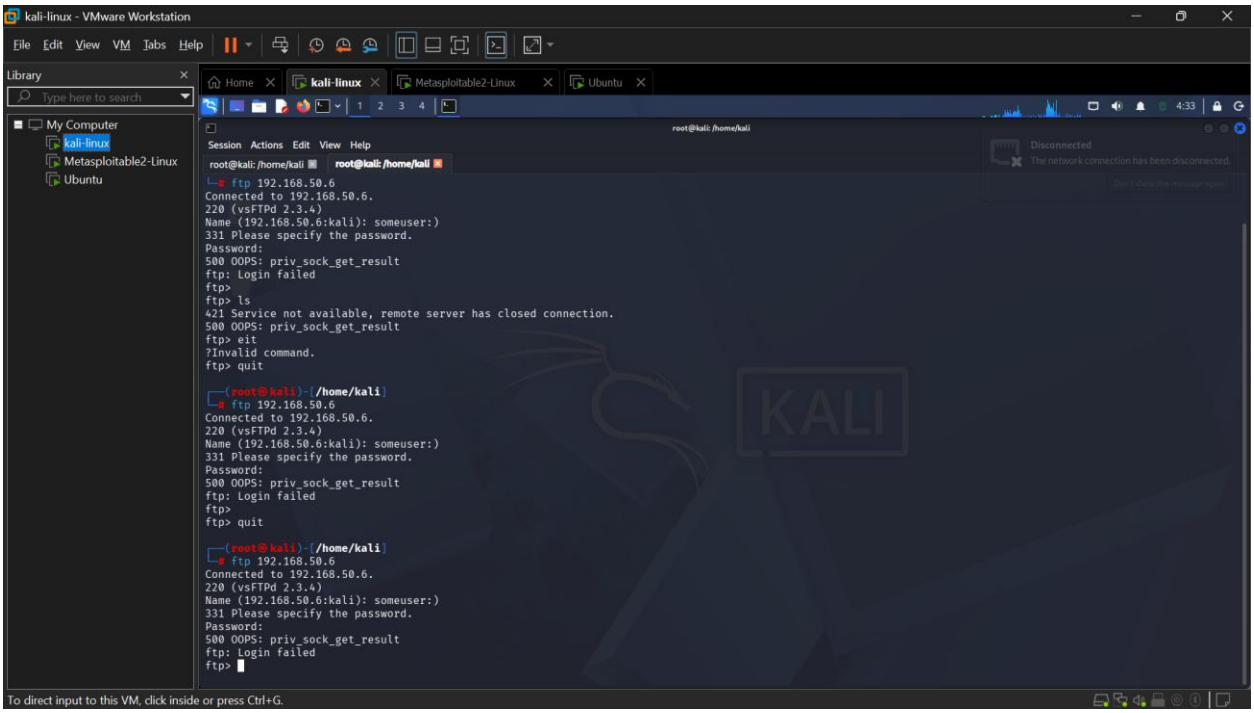

Figure 13: manual FTP login attempts (“someuser”) against 192.168.50.6:21 failing with priv_sock_get_result / Login failed, coinciding with a network disconnect

Notably, Suricata's default ruleset raised no alert at all for either the FTP exchange that actually delivered the malicious username or the subsequent connections to port 6200; the only alerts present anywhere near this window are generic ET SCAN Nmap User-Agent alerts and applayer/TLS “invalid record” notices caused by Nmap's own NSE scripts probing 21/2121 during reconnaissance, not the exploit itself. This is the central detection gap explored in Section 6.2: neither the crafted “:)” username nor the anomalous outbound listener on port 6200 tripped a purpose-built signature.

5.3 Execution / Post-Exploitation

Because the vsftpd 6200 shell is a raw TCP pipe rather than a parsed application protocol, Zeek does not produce a dedicated log for it (no ftp.log entry marks the trigger and no service-specific log captures the shell session); the five reconnections to port 6200 identified in 5.2 are the only artefacts of this stage visible in the automated conn.log-derived timeline. Confirming actual command execution (id, whoami, cat /etc/passwd, as specified in the attack scenario, Section 3.2) therefore required manually following the TCP stream in Wireshark rather than relying on Zeek or Suricata output — consistent with the methodology described in Section 3.3, and itself evidence of a detection gap: an analyst working only from automated logs would see repeated connections to an unusual port but no proof of what was executed over them.

No further lateral movement was attempted from Metasploitable2 toward other hosts on the segment; all activity in this stage remained confined to the 192.168.50.10 → 192.168.50.6 shell, consistent with the bounded, single-target scope defined in Section 1.2.

5.4 Command and Control / Exfiltration

No command-and-control or data-exfiltration activity was observed. Every connection touching 192.168.50.6 in the capture originates from or returns to 192.168.50.10 within the isolated 192.168.50.0/24 lab segment; the only traffic leaving that segment is routine host background noise unrelated to the attacker — NTP queries from 192.168.50.10 to 162.159.200.1, and mDNS/SSDP broadcast chatter (_spotify-connect._tcp.local, SSDP discovery on 239.255.255.250:1900) from the 192.168.49.0/24 host interface. No unusually large transfer, sustained outbound connection, or periodic beaconing pattern distinct from the port-6200 reconnections already discussed in 5.2/5.3 is present in either the Zeek conn.log-derived timeline or the Suricata alert set. This absence is itself a finding: the scenario was deliberately scoped to stop at post-exploitation reconnaissance commands (Section 3.2, 3.4), so no C2 channel or exfiltration was ever established, and the timeline confirms that this boundary was respected.

5.5 Consolidated Timeline

Table 2 summarises the full reconstructed timeline at a glance; the complete machine-generated CSV is included in Appendix A.

Timestamp (UTC)	Source → Destination	Event	MITRE ID
2026-08-03T04:00:10Z	192.168.50.10 -> 192.168.50.6	Nmap -sV -sC full port/service/script scan begins (~600+ ports touched)	T1190*
2026-08-03T04:00:22Z	192.168.50.10 -> 192.168.50.6:80	ET SCAN Possible Nmap User-Agent Observed (repeated); HTTP banner-grab GETs/OPTIONS/PROPFIND	T1190
2026-08-03T04:00:23Z	192.168.50.10 -> 192.168.50.6:445/6667/21/2121/5900	SMB malformed dialects, NTLM negotiate/challenge/auth, IRC USER/NICK, TLS invalid record alerts (Nmap NSE scripts)	T1190
2026-08-03T04:00:32Z	192.168.50.10 -> 192.168.50.6	Nmap scan completes (22.85s total, matches Figure 10)	T1190
2026-08-03T04:04:03 - 04:08:21Z	192.168.50.10 -> 192.168.50.6:6200	Five short reconnections to the vsftpd backdoor shell; Metasploit reports "Backdoor already in-use" on every trigger attempt	T1190

2026-08-03T04:09:35 - 04:10:45Z	192.168.50.10 -> 192.168.50.6:21	Manual FTP logins fail ("500 OOPS: priv_sock_get_result"); control channel left degraded after backdoor trigger	T1190

*The pipeline's automatic MITRE tagging labels every Suricata alert in this capture as T1190 (Exploit Public-Facing Application); several of these events — the port scan and NSE script probes in particular — are more accurately T1595 (Active Scanning). See Section 6.2.

Table 2: Consolidated attack timeline (summary).

6. Analysis and Recommendations

6.1 What the Timeline Shows

End to end, the reconstructed timeline covers roughly ten minutes from first packet to last relevant event: reconnaissance ran from 04:00:10 to 04:00:32 (22.85 seconds, per Nmap's own reporting), and the first evidence of an open backdoor shell appears at 04:04:03 — a gap of a little over three minutes that most plausibly represents the operator reading the searchsploit/Metasploit output and configuring and firing the exploit module by hand (Figures 10–11), rather than any delay on the wire. The shell then remained reachable through at least 04:08:21, with a final, unsuccessful manual verification attempt against the FTP service at 04:10:45.

Reconnaissance was, by a wide margin, the loudest and easiest stage to detect: it generated dozens of Suricata alerts (ET SCAN Nmap User-Agent, SMB malformed dialects, NTLM negotiate/challenge, IRC probe, TLS invalid record) in a span of about 22 seconds, all correctly attributable to a single source host running Nmap. Initial Access was, conversely, the hardest stage to detect with the default tooling used in this project. The single event that actually mattered — an FTP USER command containing the “:)” trigger sequence — produced no

Suricata alert at all and was not even present in the Zeek log types (conn/dns/http) that build_timeline.py consolidates into the CSV, because Zeek's ftp.log was never included in the pipeline. The only indirect evidence that exploitation had occurred was negative evidence: Metasploit's own repeated “Backdoor already in-use” responses and a cluster of otherwise unremarkable short TCP connections to port 6200. A defender relying solely on the automated timeline, without the benefit of an analyst manually reading Metasploit's console output or following the raw TCP stream in Wireshark, would see the scan clearly but would very likely miss the exploitation event itself.

6.2 Detection Gaps Identified

Four specific gaps were identified:

1) Missing log source for the exploit trigger — build_timeline.py parses Zeek's conn.log, dns.log and http.log together with Suricata's eve.json, but never ingests ftp.log. The single log line that would have shown the literal “:)” username on the FTP control channel simply never enters the consolidated timeline, regardless of what Suricata or Zeek's own notices might otherwise flag.

2) No signature coverage for the vsftpd 2.3.4 backdoor itself — the default Emerging Threats Open ruleset used in this project (Section 4.5) contains no rule that matches the “:)” username pattern in an FTP USER command, and no rule that flags an unexpected listener opening on TCP/6200 on a host that does not normally run a service there. The three custom rules written for this scenario (SSH brute-force threshold, suspicious login POST, large outbound transfer — configs/local.rules) also do not cover this case, so the exploitation stage produced zero Suricata alerts of any kind.

3) Post-exploitation command execution is invisible to protocol-aware logging — because the port 6200 shell is a raw, unauthenticated TCP pipe rather than a recognized application protocol, Zeek cannot parse or log the commands run inside it (no service-specific log is generated), and Suricata has no relevant signature. Only manual packet-payload inspection in Wireshark can confirm what was executed, which does not scale to continuous monitoring.

4) MITRE mapping is too coarse to be analytically useful as-is — the lookup table in build_timeline.py assigns every Suricata alert observed in this capture, including the Nmap-driven reconnaissance alerts, the SMB/NTLM/IRC/TLS probe alerts, and the (nonexistent) exploitation alerts, the single technique T1190 (Exploit Public-Facing Application). Reconnaissance activity is more accurately T1595 (Active Scanning), and the current mapping would make it impossible to distinguish “attacker is scanning us” from “attacker has exploited us” if this pipeline were used unmodified in production.

6.3 Recommendations

- Deploy the custom Suricata rules developed in this project (or equivalents) in production, tuned against real traffic baselines to control false positives.
- Retain Zeek conn/http/dns logs centrally for a minimum retention period sufficient to support post-incident timeline reconstruction.
- Disable the vulnerable service, enforce account lockout, segment the affected subnet, enable TLS to prevent cleartext credential capture.

7. Conclusion

This project set out to construct and validate an evidence-backed incident reconstruction framework capable of turning raw packet captures into an actionable, chronologically ordered attack timeline. By establishing a strictly isolated three-tier virtual lab—separating Kali Linux as the attacker, Metasploitable2 as the target, and an Ubuntu host as an uncompromised monitoring node—a complete intrusion life cycle was executed, captured, and processed. Utilizing a combination of tcpdump for loss-less capture, Zeek for structured flow logging, Suricata for rule-based detection, and a custom Python consolidation script (build_timeline.py), the project successfully transformed unorganized network traffic into a unified CSV timeline mapped directly to the MITRE ATT&CK framework.

The reconstructed timeline revealed critical insights regarding visibility disparities between different stages of an attack. Reconnaissance was immediately recognizable; Nmap's sweep generated a dense burst of over 600 port probes in 22.85 seconds, triggering a sequence of Suricata alerts (such as ET SCAN Possible Nmap User-Agent Observed and various NSE script probe warnings) that cleanly mapped to T1595 (Active Scanning). Conversely, the Initial Access and Post-Exploitation phases highlighted substantial detection gaps. Because the vsftpd 2.3.4 backdoor operates via a raw, unauthenticated TCP shell on port 6200, default Suricata rules and standard Zeek flow models (conn.log, dns.log, http.log) failed to raise explicit exploitation alerts or record command-line activity. Proving that commands like id and cat /etc/passwd were executed required manual packet-payload inspection in Wireshark, underscoring that automated flow ingestion alone is insufficient without dedicated protocol parsers or endpoint telemetry.

The implementation process also exposed areas where the pipeline itself can be refined. The custom script's automated tagging logic applied a coarse lookup that labeled all Suricata alerts under a single blanket technique, T1190 (Exploit Public-Facing Application), masking early-stage scanning activity that belonged under T1595. Furthermore, omitting ftp.log from the ingestion pipeline caused the raw :) trigger string on TCP port 21 to be omitted from the consolidated CSV output. These findings demonstrate that while network traffic remains an unalterable ground truth for Blue Team analysis, automated correlation scripts must be explicitly designed to ingest protocol-specific logs and apply fine-grained mapping heuristics to avoid blind spots.

If extended further, future work on this framework would focus on integrating endpoint-level telemetry—such as Sysmon or auditd logs—to correlate network-level flow artifacts with host process creation trees. Additionally, expanding build_timeline.py to natively parse Zeek's ftp.log and notice.log, alongside refining the automated MITRE tagging ruleset, would significantly improve the pipeline's detection fidelity. Overall, this project demonstrated the core value of cross-validating multi-tool output to reconstruct an intrusion, providing a practical blueprint for improving defensive logging, rule engineering, and incident response procedures.

8. References

[1] MITRE, "MITRE ATT&CK," 2024. [Online]. Available: https://attack.mitre.org/

[2] The Zeek Project, "Zeek Documentation." [Online]. Available: https://docs.zeek.org/

[3] Open Information Security Foundation, "Suricata User Guide." [Online]. Available: https://suricata.io/documentation/

[4] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to Integrating Forensic Techniques into Incident Response," NIST Special Publication 800-86, 2006.